# Ubuntu-BTOP From Source

| | |
|---|---|
| Created by | Kei Farouk |
| Category | ubuntu |
| Last updated time | @27 March 2026 02:27 |

# Compiling btop from Source – Notes

## Step 1: Check Prerequisites

Before anything, verify that `git` and the build tools are available.

```
git --version
make --version
g++ --version
```

- **git** – version control tool used to download source code from GitHub
- **make** – a build automation tool that reads a `Makefile` to know how to compile the project
- **g++** – the C++ compiler that actually turns source code into a runnable program

---

## Step 2: Install Missing Tools

Ubuntu 24 ships with g++ 13, but btop v1.4.6 requires C++23 features (specifically `std::ranges::to`) that are only fully supported in **g++ 14**.

```
sudo apt update
sudo apt install git make g++-14
```

- **sudo** – runs the command as a superuser (administrator)
- **apt** – Ubuntu's package manager, used to install software

- **g++-14** – the specific version of the C++ compiler we need

---

## Step 3: Clone the btop Repository

Navigate to where you want to store the project, then clone it.

```
mkdir ~/apps
cd ~/apps
git clone --recursive https://github.com/aristocratos/btop.
git
cd btop
```

- **mkdir** – makes a new directory
- **git clone** – downloads a copy of the repository from GitHub
- **-recursive** – also downloads any submodules (nested dependencies) the project relies on

---

## Step 4: Compile btop

Run `make`, explicitly telling it to use g++-14 as the compiler.

```
make CXX=g++-14
```

- **make** – reads the `Makefile` and compiles all the source files
- **CXX=g++-14** – overrides the default compiler with g++ version 14
- This step takes about 1-2 minutes — you'll see a progress % as each file compiles
- The output binary ends up at `bin/btop`

---

## Step 5: Install btop

Install btop into your local user directory (no sudo needed).

```
make install PREFIX=~/.local
```

- **make install** – copies the compiled binary to the install location

- **PREFIX=~/.local** – installs to your home directory under `.local/` instead of system-wide
- The binary ends up at `~/.local/bin/btop`

---

## Step 6: Add to PATH (Create the Shortcut)

Right now you'd have to type the full path `~/.local/bin/btop` every time. Adding it to your `PATH` lets you just type `btop` from anywhere.

```
echo 'export PATH=$PATH:~/.local/bin' >> ~/.bashrc
source ~/.bashrc
```

- **PATH** – an environment variable that tells your shell where to look for commands
- **echo '...' >> ~/.bashrc** – appends the export line to your shell config file so it persists across sessions
- **source ~/.bashrc** – reloads the config file in the current session so the change takes effect immediately

---

## Step 7: Verify

```
btop
```

You should see the btop resource monitoring interface launch. 🎉

---

## Quick Reference

| Command | What it does |
|---|---|
| `sudo apt install <pkg>` | Install a package |
| `git clone --recursive <url>` | Clone a repo with submodules |
| `make CXX=g++-14` | Compile using a specific compiler |
| `make install PREFIX=~/.local` | Install to home directory |
| `echo '...' >> ~/.bashrc` | Append to shell config |
| `source ~/.bashrc` | Reload shell config |

### Some Notes On Source

When you do `sudo apt install btop`, you're grabbing a **pre-compiled binary** — someone else already went through the whole compilation process, packaged the result, and put it on Ubuntu's servers. Your machine just downloads and unpacks it. Fast and easy, but you get whatever version and configuration they decided on.

When you **compile from source**, you're starting from the raw C++ code and building the program yourself on your machine. This matters for a few reasons:

**You get the latest version.** Package managers often lag behind. Ubuntu's apt had btop 1.2.13 while the source had 1.4.6 — that's a big gap!

**It's optimized for your machine.** A pre-built binary has to work on everyone's hardware, so it's compiled conservatively. When you compile yourself you can add flags that target your specific CPU architecture, squeezing out more performance. This is huge in HPC.

**You control the configuration.** Want to enable or disable certain features? Change how it handles GPU support? You can tweak the build before compiling.

**You can modify the code.** If you find a bug or want a feature, you can change the source and rebuild. This is how open source contributions work.

In HPC specifically, almost everything gets compiled from source — MPI libraries, HPL, scientific tools — because that extra performance from machine-specific optimization actually matters at scale. So what you did today with btop is genuinely the foundation of how HPC software deployment works!

# Let’s Install a Game From Source!

*Adapted from the nSnake GitHub repository by alexdantas*

## Introduction

In this tutorial you will compile **nSnake** — a classic snake game with a textual interface — from source on your Ubuntu server VM. This builds on what you learned compiling btop, but introduces a new concept: **dependencies**. nSnake relies on a library called `ncurses` to render its visuals in the terminal, and you'll need to install that before you can build.

By the end you'll have a fully playable snake game running in your terminal, compiled entirely by you. 🐍

---

## Prerequisites

- Ubuntu server VM (CLI only)
- SSH access to your VM
- `git` and `make` installed
- A C++ compiler (`g++`)

---

## What is ncurses?

Before we start — nSnake's only dependency is **ncurses**, a C library that lets programs draw and update text on the terminal screen (think: boxes, colors, cursor movement). It's what powers btop's UI too! We need the **dev** (development) version, which includes the header files needed to compile programs that use it.

---

## Instructions

### Step 1: Install Dependencies

```
sudo apt update
sudo apt install libncurses-dev
```

- **libncurses-dev** – the ncurses development library. Without this the compiler won't know how to build the game's terminal graphics.

---

### Step 2: Clone the Repository

Navigate to your apps folder and clone the nSnake source code.

bash

```
cd ~/apps
git clone --recursive https://github.com/alexdantas/nSnake.
git
cd nSnake
```

- **git clone --recursive** – downloads the source code along with any submodules
- The project is written in C++, similar to btop

---

## Step 3: Compile nSnake

```
make
```

This is much faster than btop — nSnake is a simpler project so it should compile in just a few seconds. You'll see each file being compiled one by one.

---

## Step 4: Test Before Installing (Optional but fun!)

You can run the game directly from the build folder before installing it:

bash

```
make run
```

This is a handy trick — many projects support `make run` so you can test a build before committing to installing it. Give it a quick play! Use arrow keys to move and `q` to quit.

---

## Step 5: Install nSnake

```
make install PREFIX=~/.local
```

- **PREFIX=~/.local** – installs the game into your home directory, no `sudo` needed
- The binary goes to `~/.local/bin/nsnake`
- The game levels get installed to `~/.local/share/games/nsnake/levels`

---

## Step 6: Copy the Game Levels

nSnake looks for its levels in a specific place. Copy them over so the game can find them:

```
mkdir -p ~/.local/share/nsnake
cp -r ~/.local/share/games/nsnake/levels ~/.local/share/nsn
ake/
```

- **mkdir -p** – creates the directory and any missing parent directories
- **cp -r** – copies a directory and all its contents recursively

---

### Step 7: Add to PATH

If you haven't already from the btop tutorial, add `~/.local/bin` to your PATH:

```
echo 'export PATH=$PATH:~/.local/bin' >> ~/.bashrc
source ~/.bashrc
```

---

### Step 8: Play!

```
nsnake
```

You should see the nSnake menu appear. Use arrow keys to navigate and start a game. 🎮

---

## Quick Reference

| Command | What it does |
|---|---|
| `sudo apt install libncurses-dev` | Install the ncurses dependency |
| `make run` | Test the build without installing |
| `make install PREFIX=~/.local` | Install to home directory |
| `mkdir -p` | Create directory and parents if missing |
| `cp -r` | Copy a directory recursively |

---

## Reflection Prompts

After completing the tutorial, discuss with your team:

1. What is the difference between `libncurses5` and `libncurses-dev`? Why do we need the `dev` version to compile?
2. Why might you want to run `make run` before `make install`?
3. What does it mean for a project to have a dependency, and how does that relate to HPC software stacks?

---

#### 1. libncurses5 vs libncurses-dev

`libncurses5` is the runtime library — it's what a program needs to *run* if it already uses ncurses. Most systems have this installed by default. `libncurses-dev` is the

development version, which includes the **header files** (`.h` files) that the compiler needs to understand ncurses functions while *building* the program. Think of it like this: you need the dev version to bake the cake, but only the regular version to eat it.

**2. make run before make install**

It's just good practice — you verify the build actually works before scattering files across your system. If something is broken or the game isn't what you expected, you haven't committed to anything yet. In serious HPC work this matters a lot when installing to shared systems where other people depend on the software.

**3. Dependencies and HPC software stacks**

A dependency is any external library or tool your software needs to function. In HPC this gets really deep — something like HPL (the benchmark used in the CHPC competition) depends on an MPI library for parallel communication, which depends on your network drivers, which depends on your OS kernel version. This chain is called a **software stack**. Managing it correctly is one of the hardest parts of HPC — one wrong version of one library can break everything. Tools like **EasyBuild** and **Spack** exist specifically to manage this in HPC environments, and you'll likely encounter them in the competition!

---

# Conclusion

You've just compiled and installed a fully playable terminal game from source! Beyond the fun, this tutorial reinforces key skills: managing dependencies, understanding the compile-install workflow, and working with `make`. In HPC environments, almost every tool you use — MPI, HPL, scientific libraries — follows this exact same pattern. You're building the muscle memory now!